

Indian Institute of Engineering Science and Technology, Shibpur
BTech CST 4th Semester Final Examination, May 2026

Programming Paradigms (CS-2203)

Full Marks: 50

Time: 3 Hours

Answer all questions

All parts of the same question must be answered together

1. Consider the following class definition that represents a Smart Fan. Every time the fan is switched on, it resumes its last set speed and mode.

```
class SmartFan {
    bool isOn; // true if ON, false if OFF
    int speedLevel; // range: 0 to 5
    int mode; // 1 = normal, 2 = sleep, 3 = turbo

    // In normal mode, any speed level can be set.
    // Where as in sleep mode, speed level is <= 2
    // and in Turbo mode, speed level is >= 4.

public:
    /* Member Functions */
};
```

Develop the following points using the C++ programming language. Use appropriate C++ features wherever applicable.

- Provide a constructor for the class that initializes all instance variables with valid default values.
- Declare appropriate setter and getter methods that allow modification and access of the member variables through an object.
- Define the setter and getter methods. Ensure that invalid values (e.g., speed outside 0-5 or incorrect mode) are not assigned.
- Write a `main()` function to demonstrate the basic operations of the Smart Fan, such as turning it on/off, changing speed, and switching modes.

[3 + 3 + 5 + 4 = 15]

2. Answer any 1 of the following questions

- Consider a scenario where a device called 'CamTune' can perform multiple functions, such as taking photos and playing music. You are required to implement this in C++ using multiple inheritance. Create two base classes: Camera with a function `capture()` that prints "Photo captured", and MusicPlayer with a function `play()` that prints "Music playing". Then create a derived class, CamTune, that inherits from both Camera and MusicPlayer classes. Add a member function `useDevice(...)`

in CamTune that calls both `capture()` and `play()`. In `main()`, create a CamTune object and call `useDevice(...)`.

Also modify both base classes by adding a function `powerOn()` that prints "Camera powered on" in Camera and "Music Player powered on" in MusicPlayer. Attempt to call `powerOn()` using the CamTune object in the `main()`.

- b) Write a C++ program to simulate an employee salary system using inheritance and polymorphism. Create a base class `Employee` that contains a data member `salary` and a virtual function `calculateSalary()`.

Then derive two classes, `FullTime` and `PartTime`, from the base class. The `FullTime` class should override the `calculateSalary()` function to return a fixed salary, while the `PartTime` class should override it to calculate salary using the formula ($\text{hours} \times \text{rate}$).

In the `main()` function, use a base class pointer to demonstrate runtime polymorphism by calling the `calculateSalary()` function for multiple `FullTime` and `PartTime` objects.

[1x10 = 10]

3. Answer any 5 from the following questions (any example code used should be only in C++ programming language)

- (a) What is the purpose of declaring a destructor as virtual in C++? Is it always advisable to make the destructor of a base class virtual?
- (b) Define "Functional Independence" in module design. How are coupling and cohesion related to achieving functional independence?
- (c) Why is use case factoring required in UML use case diagrams? Explain use case factoring using `<<include>>` and `<<extend>>` relationships with examples.
- (d) Explain the difference between aggregation and composition in object-oriented class relationships. Provide a suitable example to illustrate each.
- (e) What is a constructor initialization list in C++? Explain situations where using an initialization list is mandatory, with examples.
- (f) What is multilevel inheritance? Describe its concept and demonstrate it with a clear C++ code example.
- (g) Explain various techniques of operator overloading in C++ using examples.

[5x5 = 25]